

Forward Deployed Engineer Case

Building a Canteen Menu Planning Engine

Context

You are joining the Forward Deployed Engineering team at Heyra. One of our customers is a **university canteen**, and they want help planning their weekly lunch menu. They serve two tracks at lunch — one meat, one vegetarian — but for this pilot we're starting with a single slice to get something working in front of the customer fast: **the vegetarian track**, Monday through Friday. Today the canteen team plans this by hand; we want to explore whether an LLM-driven engine can do most of the heavy lifting from a product catalogue.

Your job is to build a proof of concept that, given the canteen's product catalogue and a small set of constraints, produces **one week of vegetarian mains: 5 days = 5 main dishes**, each grounded in **real products from the catalogue**.

This is a forward-deployed exercise: we care about a working, demoable slice and your ability to explain the trade-offs you made, more than about a polished architecture. How you get there is up to you.

The Data

You will work with a curated catalogue of plausible food-service ingredients, preprocessed into a single `products` table.

- `products.duckdb`, a [DuckDB](#) database file. Zero-setup, queryable from the `duckdb` CLI or Python.
- `products.csv`, the same rows in CSV, if you'd rather use pandas or SQLite.

Use whichever fits your stack; both contain the same data.

Schema

A single table, `products`, with the following columns:

Column	Type	Description
<code>product_id</code>	INTEGER	Stable product identifier
<code>product_name</code>	TEXT	Product name (English)
<code>ingredient_group</code>	TEXT	Free-text ingredient group (29 distinct values)
<code>cost_per_100g_eur</code>	NUMERIC	Cost in € per 100g, synthesized , see caveats
<code>energy_kcal_per_100g</code>	NUMERIC	Energy per 100g, sampled deterministically per ingredient
<code>volume_ml_per_100g</code>	NUMERIC	Liquid volume per 100g, non-null only for liquid groups

dietary_class	TEXT	One of vegan, vegetarian, meat
allergen_gluten	INTEGER	0 or 1
allergen_nuts	INTEGER	0 or 1
allergen_dairy	INTEGER	0 or 1
is_available	INTEGER	0 or 1; only is_available=1 rows should be considered available

Loading it

```
# DuckDB CLI
duckdb products.duckdb
# SELECT COUNT(*) FROM products;
```

```
# Python
import duckdb
con = duckdb.connect("products.duckdb")
con.sql("SELECT * FROM products WHERE dietary_class = 'vegan' LIMIT 5").show()
```

```
# Or just CSV + pandas
import pandas as pd
df = pd.read_csv("products.csv")
```

What's in there

- **3,137 rows** across **29 ingredient groups**; roughly 81% are `vegan` or `vegetarian`, so the vegetarian track has plenty to work with.
- `volume_ml_per_100g` is **null for non-liquids** — that's expected, not a data error. **Only `is_available=1` rows should be considered available.**
- **Enriching the catalogue is fair game.** If you find it useful to derive or augment the data in some way (normalising ingredient groups, tagging products, etc.), feel free to do so.

Your Task

Build a system that produces **one week of vegetarian canteen mains**: Monday through Friday, one main dish per day, grounded in the supplied product catalogue. Note that the catalogue gives you *products*, not dishes. **Going from a bag of ingredients to a coherent set of dishes is part of the work.**

The shape is deliberately broad. You decide how to break the problem up, what the LLM does vs. what deterministic code does, and how the pieces fit together. We want to see your **design thinking**, not a specific architecture.

What "done" looks like

A weekly plan covering Mon–Fri with one vegetarian main per day, where every dish is:

- grounded in real products from the catalogue,

- on the vegetarian track: every dish uses only `dietary_class` of `vegan` or `vegetarian` (no meat),
- internally consistent: quantities are positive, the dish makes sense as a main, allergen flags are correct.

What counts as a "main dish"? Treat a main as a reasonably complete plate: a carb base (e.g. rice, pasta, potatoes, bread), a protein component (legumes, tofu, eggs, dairy, etc.), some vegetables, and ideally a sauce or dressing to tie it together. You don't need to enforce this rigidly but feel free to use it as a guide for what a sensible, balanced main looks like, and decide for yourself how strictly to validate it.

Requirements

1. **Grounded in products.** Your system must discover ingredients by querying the catalogue: semantic search, filtered SQL, keyword filter, however you like. It should not invent products.
2. **Structured output + validation.** The weekly plan must come back as structured JSON. Validate that every `product_id` actually exists, that quantities make sense, and that the track constraint (no meat) holds. The right place to draw the line between "LLM does it" and "code checks it" is part of what we're assessing.
3. **Graceful failure.** When the LLM hallucinates a product, the output JSON is malformed, or the catalogue can't fill a slot, your system should fail in a way that's actually useful to the user, not silent or cryptic.
4. **A readable summary.** Produce a simple weekly summary — total cost and a calorie/allergen overview is plenty — in any human-readable form (printed table, Markdown, a small web view). It just needs to be something you could show the canteen team in a demo.

Looking ahead, extensions to think about (not to build)

You don't need to implement these, but be ready to talk through how you'd approach each in the interview. A couple of slides is enough.

1. **Variety across the week.** The five vegetarian mains shouldn't be near-duplicates of each other, e.g. two pasta-with-tomato-sauce variants. How would you define "near-duplicate", and where would that check live?
2. **"Swap this dish."** A follow-up that replaces one day's dish with an alternative on request — including the case where the chosen product turns out to be unavailable (`is_available = 0`).
3. **Evaluation.** How would you build a lightweight way to check whether the output is correct? Think about test cases with expected properties, automated checks on structural validity, or a structured manual review. We care about how you think about correctness more than coverage.

Constraints and Guidance

- **Time budget: 2–4 hours.** This is a focused, forward-deployed exercise, not a weekend project. A working slice you can demo beats a sprawling, half-working one. If you go over, stop and write down what you'd do next.
- Use any LLM API you prefer (OpenAI, Anthropic, open-source, etc.). Free trial credits work fine.
- Use any language and framework, whatever lets you move quickly.

- **A small web app is encouraged** but not required: a single page where you can enter constraints, watch the engine run, and inspect the output makes the customer demo much easier. Streamlit, Gradio, FastAPI + plain HTML, pick what you can ship fast.
- AI coding tools are welcome and encouraged. We care about the end result and your decisions, not whether you typed every line.

Deliverables

1. **Working code**, with a short README covering setup only.
2. At the interview, expect a **short demo** of the engine in action, followed by a **short walkthrough** of your decisions: where the LLM helps vs. where deterministic code does, how you handle failure, what you'd improve with more time, and how you'd tackle the "looking ahead" extensions.

How We Evaluate

We are not looking for a production-ready product. We want to understand how you think, build, and communicate.

Criteria	What we look for
Pragmatism & Speed	Did you ship a working slice in the budget? Sensible trade-offs over heroic over-scoping?
System Design	Is the boundary between LLM and deterministic code (validation, search, assembly) clean and easy to follow?
Error Handling	When the LLM hallucinates a product or returns malformed JSON, does the system fail gracefully and informatively?
Communication	Can you clearly explain your decisions and trade-offs — the way you would to a customer on-site?

Questions?

If anything is unclear, don't hesitate to reach out to your point of contact at Heyra. We'd rather you ask than make assumptions.

Good luck, we're looking forward to seeing what you build.